

FULL STACK DEVELOPMENT

FASE 4 | AULA 05 -

SINGLE SPA - PARCELS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	8
REFERÊNCIAS.....	9

EMSE

O QUE VEM POR AÍ?

Nesta aula, vamos explorar um conceito fundamental para quem trabalha com Single SPA: os “parcels”. Se você já está familiarizado com microfrontends, vai descobrir que os parcels oferecem uma forma ainda mais granular de modularizar sua aplicação, permitindo uma flexibilidade impressionante na reutilização de componentes.

Vamos começar entendendo o que exatamente são os parcels e como eles se diferenciam dos microfrontends tradicionais. Enquanto os microfrontends geralmente representam seções maiores da aplicação, como páginas inteiras ou funcionalidades completas, os parcels são mais leves e versáteis. Eles podem ser pensados como minimicrofrontends que você pode injetar em qualquer parte da sua aplicação, independentemente do framework ou tecnologia utilizada.

Além de explorar o conceito, vamos ver na prática como configurar e registrar parcels dentro do Single SPA. Vamos te guiar passo a passo na criação de um parcel, desde sua configuração básica até o registro no root config, garantindo que você entenda como integrá-los corretamente em sua arquitetura de microfrontends.

Na sequência, veremos como utilizar e integrar parcels de maneira eficaz em uma aplicação existente. Os parcels são ideais para componentes reutilizáveis, como um botão de login ou um widget de notificação, que você pode querer usar em várias partes diferentes da aplicação sem ter que duplicar código ou criar dependências pesadas. Vamos examinar exemplos práticos de como implementar essas pequenas, mas poderosas, peças de funcionalidade e como elas podem ser integradas perfeitamente ao seu projeto.

HANDS ON

Nesta aula, vamos mergulhar no mundo dos parcels no Single SPA e descobrir como essa funcionalidade pode transformar a maneira como você desenvolve e reutiliza componentes dentro de uma arquitetura de microfrontends. Dividida em dois vídeos, esta aula vai te guiar na configuração e integração de parcels, mostrando como aproveitar ao máximo essa ferramenta.

Começaremos entendendo o conceito de parcels e como eles se diferenciam dos microfrontends convencionais. Vamos do passo a passo de como configurar um parcel no Single SPA, sua criação, até o registro no root config. Os parcels são como mini-microfrontends que você pode utilizar em qualquer parte da aplicação, proporcionando uma flexibilidade enorme na forma como você modulariza seu projeto. Assista esta aula para aprender como configurar e registrar parcels e dar mais modularidade à sua aplicação.

Vamos explorar também, a implementação prática e integração dos parcels. Vamos te mostrar como usar parcels para criar componentes reutilizáveis que podem ser integrados em diferentes partes da aplicação sem esforço. Isso permite que você mantenha seu código organizado e eficiente, evitando a duplicação de funcionalidades e garantindo uma arquitetura mais coesa e sustentável. Você aprenderá como utilizar parcels de maneira eficaz e integrá-los perfeitamente em sua aplicação existente.

SAIBA MAIS

Design tokens

Quando falamos de microfrontends, um dos grandes desafios é manter a consistência visual em uma aplicação composta por várias partes independentes, cada uma possivelmente desenvolvida por diferentes equipes. É aqui que entram os design tokens, que podem ser a chave para garantir que todos os microfrontends compartilhem uma identidade visual coerente.

Design tokens são, basicamente, valores abstratos e reutilizáveis que representam as decisões de design de uma marca ou produto. Eles incluem coisas como cores, tipografia, espaçamentos, tamanhos de fontes, bordas e muito mais. Em vez de codificar esses valores diretamente nos estilos de cada microfrontend, os design tokens centralizam essas decisões em um único conjunto de definições que pode ser aplicado em toda a aplicação.

Imagine que você está trabalhando em uma aplicação que utiliza diferentes microfrontends para várias partes do sistema. Sem design tokens, cada microfrontend poderia definir suas próprias cores, fontes e espaçamentos, o que rapidamente levaria a uma fragmentação visual. Isso significa que, mesmo que os microfrontends sejam tecnicamente corretos, a experiência do usuário pode parecer desorganizada ou incoerente. Um botão em uma parte da aplicação pode ter um visual completamente diferente de outro botão em uma seção distinta, o que pode confundir os usuários e prejudicar a identidade da marca.

Com design tokens, você evita esse problema. Por exemplo, ao invés de cada equipe escolher sua própria paleta de cores, os tokens fornecem uma paleta centralizada, que todos deverão usar. Se você precisar mudar a cor principal da sua aplicação, basta atualizar o token correspondente, e essa mudança se propaga automaticamente por todos os microfrontends, mantendo a consistência e facilitando a manutenção.

Os design tokens são especialmente úteis em microfrontends, porque eles permitem que cada parte da aplicação seja desenvolvida de forma independente, mas, ainda assim, permitem que a aplicação siga as mesmas diretrizes de design. Eles

funcionam como uma linguagem comum que todos os microfrontends compreendem, garantindo que o resultado será coeso.

Além disso, os tokens são geralmente definidos em um formato que pode ser facilmente integrado a diferentes tecnologias e frameworks. Isso é crucial em microfrontends, onde você pode ter uma parte da aplicação feita em React, outra em Angular e outra em Vue. Como os tokens são agnósticos em relação ao framework, eles podem ser aplicados em qualquer ambiente, mantendo a consistência visual em toda a aplicação.

Por fim, o uso de design tokens também facilita a escalabilidade e a evolução da aplicação. À medida que sua aplicação cresce e novos microfrontends são adicionados, os tokens garantem que o novo conteúdo seguirá as mesmas diretrizes visuais dos microfrontends existentes. Isso também significa que as mudanças de design podem ser implementadas de forma rápida e eficaz, sem precisar refatorar cada parte da aplicação manualmente.

Single SPA: o ambiente de desenvolvimento

Quando se trata de trabalhar com o Single SPA, o ambiente de desenvolvimento é uma peça fundamental para garantir que sua experiência seja eficiente, produtiva e, claro, agradável. Como o Single SPA é uma ferramenta que permite a integração de múltiplos microfrontends em uma única aplicação, é crucial que o ambiente de desenvolvimento seja bem estruturado para lidar com a complexidade e a modularidade que essa abordagem traz.

Vamos começar falando sobre a configuração inicial. No Single SPA, você geralmente começa criando um root config, que é a configuração principal da aplicação. Este arquivo é como o maestro que coordena todos os microfrontends, determinando quais módulos devem ser carregados e quando. Para que o desenvolvimento flua bem, é importante que esse root config esteja bem organizado, com as rotas, dependências e importações todas claramente definidas. Isso garante que, à medida que você adiciona novos microfrontends (ou modifica os existentes), o processo seja simples e direto.

Outra parte essencial do ambiente de desenvolvimento no Single SPA é a integração com diferentes frameworks e bibliotecas. Como o Single SPA permite que você use múltiplos frameworks dentro da mesma aplicação, tais como React, Angular,

Vue etc., é necessário garantir que o ambiente de desenvolvimento suporte bem essa diversidade. Isso significa configurar o Webpack ou outra ferramenta de build para lidar com os diferentes tipos de arquivos e compilações que esses frameworks requerem. O uso de uma configuração compartilhada, como as oferecidas pelo próprio Single SPA, pode facilitar muito esse processo, garantindo que todos os microfrontends possam coexistir sem conflitos.

Agora, vamos falar sobre o “hot-reloading”. Trabalhar em microfrontends pode ser mais complicado do que trabalhar em uma aplicação monolítica, especialmente quando se trata de mudanças em tempo real. Felizmente, o Single SPA suporta hot-reloading, o que significa que você pode ver instantaneamente as mudanças realizadas em um microfrontend, sem precisar recarregar toda a aplicação. Isso é extremamente útil, pois economiza tempo e ajuda a detectar problemas mais rapidamente.

Outro aspecto importante do ambiente de desenvolvimento é a integração com ferramentas de devtools. O Single SPA tem um conjunto de devtools que facilita a depuração e o monitoramento dos microfrontends em tempo real. Com essas ferramentas, você pode inspecionar o estado de cada microfrontend, ver quais estão carregados, verificar as comunicações entre eles e identificar quaisquer problemas que possam estar ocorrendo. Essas devtools são uma parte essencial do seu kit de ferramentas, ajudando a manter tudo sob controle, mesmo quando você está lidando com uma aplicação complexa e fragmentada.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, nos aprofundamos em um aspecto fascinante do Single SPA: os ~~parcels~~. Esses pequenos, mas poderosos componentes oferecem uma maneira ainda mais granular de modularizar sua aplicação. Enquanto os microfrontends tradicionais lidam com partes maiores da aplicação, os ~~parcels~~ permitem que você injete funcionalidades específicas, como pequenos blocos de construção, em qualquer parte do seu projeto.

Vimos sobre como os ~~parcels~~ se diferenciam dos microfrontends convencionais por serem mais leves e focados, ideais para quando é preciso um componente reutilizável que possa ser integrado de maneira fluida em diferentes contextos, sem a complexidade de um microfrontend completo.

Exploramos o processo de configuração e registro de ~~parcels~~ no Single SPA, garantindo que você saiba exatamente como criar e incorporar esses componentes em sua aplicação. Essa habilidade é essencial para aumentar a modularidade do seu código, permitindo que você reutilize funcionalidades sem criar dependências pesadas ou complicações na manutenção.

Também discutimos como os ~~parcels~~ podem ser integrados em uma aplicação existente, facilitando a reutilização de componentes como botões, menus ou qualquer outro elemento que você precise replicar em várias partes da aplicação. Isso não só otimiza o desenvolvimento, como também mantém o código organizado e fácil de escalar.

REFERÊNCIAS

SINGLE-SPA **Documentation. Videos.** Disponível em: <https://single-spa.js.org/docs/videos>. Acesso em: 14 ago. 2024.

SINGLE-SPA Documentation. **Parcels Overview.** 2024. Disponível em: <https://single-spa.js.org/docs/parcels-overview>. Acesso em: 14 ago. 2024.

SINGLE-SPA Documentation. **Shared Webpack Configs.** Disponível em: <https://single-spa.js.org/docs/shared-webpack-configs>. Acesso em: 14 ago. 2024.

EMANIP

PALAVRAS-CHAVE

Parcels. Single SPA. Modularidade.

EMENDAS

POS TECH